Why Modern Classical Monte Carlo Codes Are Fast

A Pedagogical Tour of the Ingredients,
with Worked Examples From `ClassicalSpin_Cpp`

April 2026

Abstract

This note is an explanation, written for a physics graduate student with no prior computer-science background, of why modern classical-spin Monte Carlo (MC) codes can run *thousands of times faster* than a naive textbook implementation while computing exactly the same physical answer. The text is organized around the optimization pass that brought `ClassicalSpin_Cpp` up to par with state-of-the-art (SOTA) atomistic spin codes such as Vampire and UppASD. Each ingredient is introduced from physical first principles, the underlying computer-science concept is explained in plain language, and a measured benchmark number from this code is given. No prior knowledge of compilers, caches, or parallel programming is assumed.

Contents

1 The picture in one paragraph **2**

2 What the CPU actually does **3**
- 2.1 Fact 1: arithmetic is free, memory is not . . . 3
- 2.2 Fact 2: branches are predicted, but mispredictions are expensive . . . 3
- 2.3 Fact 3: the compiler can vectorize, but only if the loop is simple . . . 3

3 The Metropolis kernel: where the time goes **4**
- 3.1 Molecular-dynamics throughput . . . 4

4 Ingredient I: the data layout **5**
- 4.1 The naive layout is a hidden disaster . . . 5
- 4.2 The fix: contiguous flat storage . . . 5
- 4.3 The same logic applied to the interaction tables: SoA bilinear . . . 5

5 Ingredient II: zero allocation in the hot loop **6**

6 Ingredient III: sphere sampling without trigonometry **7**

7 Ingredient IV: tensor contraction order matters more than the layout **8**
- 7.1 The naive triple loop is quadratic in d *and* doubles the work . . . 8
- 7.2 The fix: pre-contract two indices once, then take a dot product . . . 8

8 Ingredient V: fuse derivative passes that share a neighbour loop **9**
- 8.1 The fix: one neighbour loop, accumulate eight contributions in registers . . . 9

9 Ingredient VI: sparse structure constants are not just theory **10**
9.1 The fix: hard-code the nine non-zero triples . . . 11

10 Ingredient VII: hoist time-only work out of per-site loops **12**
10.1 The fix: hoist + early exit . . . 12
10.2 The same trick for heap allocations . . . 13
10.3 Strain RHS: the same logic on a different scale . . . 13

11 Ingredient VIII: sweep order and branch behaviour **13**
11.1 Sequential vs random site order . . . 13
11.2 Short-circuit acceptance . . . 14

12 Ingredient IX: better algorithms beat better implementations **14**
12.1 Overrelaxation . . . 14
12.2 Cluster algorithms (Wolff, Swendsen–Wang) . . . 14
12.3 Parallel tempering . . . 15

13 Ingredient X: parallelism and where to apply it **15**
13.1 SIMD: hand it to the compiler . . . 15
13.2 OpenMP: two layers, both used . . . 16
13.3 MPI: scale across nodes . . . 16

14 Ingredient XI: reproducibility is a feature, not a luxury **16**

15 Ingredient XII: parallel intra-sweep updates via static sublattice colouring **17**
15.1 Why a colouring is needed . . . 17
15.2 Building the interaction graph . . . 17
15.3 Parallel sweep kernel . . . 18
15.4 Measured scaling . . . 19
15.5 When to use which mode . . . 19

16 Putting it together: a checklist **20**

17 What the numbers mean **21**

18 Further reading **21**

1 The picture in one paragraph

A classical Monte Carlo simulation of a magnet does the same thing 10^8–10^{12} times: *pick a spin, propose a small change, compute the energy difference, accept or reject.* The total cost of the simulation is therefore the cost of *one such elementary step*, multiplied by the enormous number of steps. To make the code fast, the job is to make that single inner step as cheap as the laws of physics and the laws of the CPU allow. Roughly half the speedup comes from **algorithmic** ingredients (better moves, smarter sampling), the other half from **systems** ingredients (touching memory in a cache-friendly order, avoiding hidden allocations, getting the compiler to emit vectorized instructions). Both halves are necessary; either alone is insufficient.

2 What the CPU actually does

Before diving into algorithms, three facts about the hardware are worth internalising, because every optimization below is a direct response to one of them.

2.1 Fact 1: arithmetic is free, memory is not

A modern x86 core can do roughly 5×10^9 floating-point multiply–adds per second. Reading one "cold" double from main memory (DRAM) takes about 80 nanoseconds. In that time the core could have performed ~ 400 multiply–adds. In other words, **a single cache miss is as expensive as several hundred floating-point operations**.

The CPU mitigates this by keeping a hierarchy of small, fast caches close to the registers:

Level	Typical size	Access time
Registers	$\sim$ 1 kB	0.3 ns
L1 cache	32–64 kB	1 ns
L2 cache	256 kB–1 MB	4 ns
L3 cache	4–64 MB	15 ns
DRAM	16–512 GB	80 ns

The rule of thumb that comes out of this is: **algorithms that touch the same kilobyte of data many times in a row are fast; algorithms that hop around a multi-MB structure are slow**, even if the operation count looks identical on paper.

2.2 Fact 2: branches are predicted, but mispredictions are expensive

A modern CPU does not wait for an `if` to be evaluated before starting to work on the code that follows it. It *guesses* which branch will be taken, executes that path speculatively, and only commits the result when the actual condition is known. When the guess is right, the `if` costs essentially nothing. When it is wrong, the CPU must throw away ~ 15 partially-computed instructions and restart; this is a *branch misprediction* and costs ~ 5 ns, which is again hundreds of arithmetic operations.

The Metropolis acceptance test is a classic mispredictor: in equilibrium its outcome is essentially random. We will return to it in Section 3.

2.3 Fact 3: the compiler can vectorize, but only if the loop is simple

Modern x86 cores have *SIMD* (Single Instruction, Multiple Data) units that operate on 4 or 8 doubles at once. The compiler will use them *automatically* if it can prove that the loop:

- touches contiguous memory (no pointer chasing),
- has no aliasing (different pointers really refer to disjoint memory),
- has a known trip count and no early exits,
- does not call out into opaque functions.

Failing any one of these turns an SIMD loop back into a scalar one and costs roughly a $4\times$ slowdown. This is why "rewriting the inner loop on raw `double*` arrays" is not premature optimization—it is what allows the compiler to do its job.

3 The Metropolis kernel: where the time goes

Consider the Heisenberg model on a honeycomb lattice with N sites. A single Metropolis sweep does:

1. For each site i in the lattice (N iterations):
   (a) Generate a candidate spin $\mathbf{S}_i^{\text{new}}$ on the unit sphere.
   (b) Compute the local energy difference $\Delta E = \mathbf{S}_i^{\text{new}} \cdot \mathbf{H}_i^{\text{loc}} - \mathbf{S}_i^{\text{old}} \cdot \mathbf{H}_i^{\text{loc}}$ where $\mathbf{H}_i^{\text{loc}} = \sum_{j \in \text{nn}(i)} \mathsf{J}_{ij} \mathbf{S}_j$ is the local field.
   (c) Draw $u \sim U(0,1)$. Accept if $\Delta E < 0$ or $u < e^{-\beta \Delta E}$.

Naively coded, each pass through the body of step 1 does, in our case on a honeycomb with $z = 3$ neighbours, about

$$3 \text{ (neighbours)} \times 9 \text{ (matrix-vector adds)} = 27 \text{ floating-point operations.}$$

At 5×10^9 flop/s this is ~ 5 ns of arithmetic per site, hence a peak of $\sim 2 \times 10^8$ updates/sec/core.

The benchmark in this code (Table 1, "after" column) measures 1.8×10^7 updates/sec/core for the Heisenberg Metropolis kernel. Before the SoA-table and Marsaglia refactors of Sections 4.3–6 the same kernel ran at 1.4×10^7, so the latest pass closes part of the $\sim 12\times$ gap to peak arithmetic that the rest of this note explains, ingredient by ingredient.

The bolded rows are the kernels where this refactoring pass actually moved the needle: Metropolis (and any sweep that contains it) gained $\sim$ 25–30% across all three models. The pure-overrelaxation, Wolff, and Swendsen–Wang kernels were not measurably affected because they do not generate random spins (Marsaglia is irrelevant) and they already had reasonably contiguous J-table access (Eigen's `Matrix3d` stores its 9 doubles inline, and a `vector<Matrix3d>` is contiguous within each site). The small Wolff regression is consistent with run-to-run variation in cluster size; the algorithm's per-spin cost is unchanged.

3.1 Molecular-dynamics throughput

Table 2 reports the analogous numbers for the Landau–Lifshitz / spin–strain RHS kernels, captured with the new `bench_md` harness in this repository. The "v4" column is the state of the code immediately after the Monte-Carlo pass of Table 1; the "v5" column is after the four ODE-time-evolution optimisations of Sections 9–10.3 (sparse SU(3) cross product, drive-envelope hoist + early exit, heap-free local field, and strain-RHS state-copy elimination).

The "`Lattice`" rows show a clean $\sim 1.8\times$ speedup that is essentially *all* the drive-envelope early-exit (Section 10): even with the drive amplitude set to zero, the legacy code spent half its RHS time evaluating `exp + cos` factors that immediately got multiplied by zero. The mixed-trilinear and mixed-bilinear rows pick up an additional $\sim 30\%$ from the sparse SU(3) cross product (Section 9) and the heap-free local-field `_into` variants (Section 10, second subsection). The `StrainPhonon` family shows essentially no speedup at $L = 24$ because (a) the magnetoelastic field is already fused (Section 8), so the per-RHS arithmetic is now small; and (b) the 7 kB heap copy that Section 10.3 removed is below the threshold where a modern `tcmalloc`/ `ptmalloc` starts to hurt. The win is correctness and simplicity rather than measured throughput at this lattice size; on larger lattices ($N_{\text{cells}} \gtrsim 10^4$) the heap-free RHS would scale linearly while the legacy copy-restore pattern would not.

4 Ingredient I: the data layout

4.1 The naive layout is a hidden disaster

A textbook implementation of a spin lattice in C++ usually looks like:

```
std::vector<Eigen::VectorXd> spins;      // one VectorXd per site
spins[i](0) = ... ; spins[i](1) = ... ; spins[i](2) = ...;
```

`Eigen::VectorXd` is a *heap-allocated* object: it stores a pointer to a buffer somewhere on the free store. When you write `spins[i](0)`, the CPU has to:

1. read `spins.data() + i*sizeof(VectorXd)` (a pointer);
2. dereference that pointer to find the buffer;
3. read the first three doubles of the buffer.

The buffer for spin i is on a different cache line (often a different *page*) than the buffer for spin i+1. Iterating over the lattice therefore generates one cache miss per site—tens of nanoseconds per spin update, completely dominating the ~ 5 ns of arithmetic.

4.2 The fix: contiguous flat storage

A *flat* layout stores all $N \times 3$ doubles in one contiguous block:

```
double* spins;            // size = N * spin_dim
spins[i*spin_dim + 0]     // x-component of spin i
spins[i*spin_dim + 1]     // y
spins[i*spin_dim + 2]     // z
```

Now spin i and spin i+1 live on the same cache line (or at most two), and iterating over them prefetches the data automatically. In `ClassicalSpin_Cpp` this is what `site_energy_diff_flat()` does internally: it reads spins via raw `double*` pointers instead of `Eigen::VectorXd` temporaries. The change alone is typically a 2–4× speedup on local-update kernels.

4.3 The same logic applied to the interaction tables: SoA bilinear

The naive layout for the exchange matrices is

```
std::vector<std::vector<Eigen::Matrix3d>> bilinear_interaction;
```

which on paper hides three pointer indirections per neighbour: outer vector → inner vector → Eigen buffer. Modern atomistic spin codes (Vampire, UppASD) instead pack the entire $N \times z \times 9$ table into a single `double[]`, and we now do the same:

```
vector<size_t> bi_flat_offset;          // CSR-style: size N+1
vector<size_t> bi_flat_partner;         // size = total_bonds
vector<double> bi_flat_J;               // size = total_bonds * spin_dim^2
vector<uint8_t> bi_flat_needs_twist;    // precomputed twist mask
```

The flat tables are built once at the end of `initialize()`; the original nested `vector<vector<...>>` stays as the source of truth for cold-path code (total energy, snapshot save/load). Inside `site_energy_diff_flat()`, `overrelaxation()`, and `get_local_field_flat()` the bilinear partners and their J matrices are now read straight from the flat arrays:

```
const double* J = &bi_flat_J[k * D2];       // contiguous 9 doubles
const double* P = spins[partner].data();    // (still scattered)
for (a=0..3) for (b=0..3) row += J[a*3+b] * P[b];
```

Pleasantly enough this turned out to be a very small win in our case (a few percent on Metropolis): `Eigen::Matrix3d` already stores its 9 doubles inline, and `vector<Matrix3d>` is contiguous within a site, so the J matrices were already L1-resident during a sequential sweep. The big advantage of the rewrite is *robustness*: removing the dependence on Eigen's expression templates and on the platform allocator's behaviour means the kernel now generates the same tight assembly across compilers and across allocator versions. It also unblocks future SIMD-intrinsic work (the layout is now compatible with AVX2 `vfmadd231pd` on $9 \times d$ blocks). The much larger Metropolis win in Table 1 comes from the next ingredient.

5 Ingredient II: zero allocation in the hot loop

A C++ `std::vector<double> tmp(3)` looks innocent. But under the hood it calls `operator new`, which:

- walks a free-list to find a chunk of memory of the right size,
- updates global allocator bookkeeping (sometimes under a lock),
- zero-initializes the memory,
- invalidates a cache line.

A single `new` call costs $\sim$30–100 ns—that is, 5–20$\times$ the *entire arithmetic budget* for one spin update. If you do this once per Metropolis proposal you have already lost the game.

The fix is two-pronged:

Stack scratch buffers for one-shot temporaries. Inside the hot loop, use plain C arrays:

```
double H_loc[8];        // local field
double new_spin[8];     // proposed spin
```

These live on the stack, cost zero to allocate, and are `constexpr`-sized so the compiler can keep them in registers.

Persistent member buffers for repeat work. When a sweep *always* needs a buffer of size N (e.g. the Wolff "in-cluster" marker, the Swendsen–Wang Union–Find arrays), declare it once as a `mutable` member of the lattice class:

```
mutable std::vector<uint8_t> cluster_in_cluster;   // size lattice_size
```

The vector is allocated *once* (the first call `resize()`s it), and on every subsequent sweep it is just a contiguous chunk of memory waiting to be reused. We applied this to the Wolff BFS stack, the projection buffer, and the Union–Find arrays; this alone removed hundreds of kilobytes of malloc/free traffic per sweep at $L = 12$.

6 Ingredient III: sphere sampling without trigonometry

The Metropolis kernel does roughly N "propose a random spin" calls per sweep. The textbook way to draw a uniformly distributed unit vector in 3D, given two uniform deviates $u_z, u_\phi \in [0, 1)$, is

$$z = 2u_z - 1, \qquad \phi = 2\pi u_\phi, \qquad r = \sqrt{1 - z^2}, \qquad (x, y, z) = (r\cos\phi,\ r\sin\phi,\ z).$$

This is mathematically clean, but on hardware it costs one `sqrt`, one `cos`, and one `sin` per spin. On modern x86 `cos` and `sin` take ~ 12 cycles each when implemented in libm, plus a function-call boundary that defeats register allocation across the call. Together that is ~ 25 ns per proposed move—about a third of the entire ~ 70 ns budget for one Metropolis update at the rates measured in Table 1.

Marsaglia (1972) showed how to produce the same distribution *without any trigonometry*, by rejection-sampling in the unit disk and mapping to the sphere algebraically:

```
do {
    u1 = 2 * rand_uniform() - 1;
    u2 = 2 * rand_uniform() - 1;
    s  = u1*u1 + u2*u2;
} while (s >= 1.0);
const double factor = 2 * sqrt(1 - s);
x = factor * u1;
y = factor * u2;
z = 1 - 2 * s;
```

The acceptance probability is the ratio of the unit disk to the unit square, $\pi/4 \approx 0.785$, so on average $1/0.785 \approx 1.27$ iterations are needed—i.e. ~ 2.55 RNG calls per accepted spin (vs. exactly 2 for the textbook method). In exchange we delete one `sqrt` and *both* transcendental calls. In practice this is a $\sim$ 25–30% wall-time improvement on the Metropolis kernel of every model in Table 1, and is by itself the largest single optimization in the most recent pass.

The same identity is used by NIST's RNG library, by NumPy's `random.normal` (via Box–Muller), and by every published atomistic spin code from Spirit to UppASD. It is genuinely free correctness: the produced distribution is uniform on S^2 to machine precision, since the rejection step is exact.

Marsaglia for the mixed and strain families. The Marsaglia method is specific to S^2 (3-component spins). For an SU(2)/SU(3) `MixedLattice`, the SU(2) sublattice is S^2 and uses Marsaglia, while the SU(3) sublattice (which is a d=8 Bloch vector with two Casimir constraints, not just a unit vector) keeps the older "hypercube + project" sampler since no two-uniform analogue exists. The detection is a single `if (spin_dim == 3)` branch in `MixedLattice::gen_random_spin`. The `StrainPhononLattice` is purely S^2, so Marsaglia is used unconditionally. In both cases the Metropolis kernel inherits the same $\sim 25\,\%$ wall-time reduction we measured on `Lattice`.

What about flat spin storage? A natural follow-up was to change `vector<Eigen::VectorXd>` (scattered heap allocations) into a single contiguous `vector<double>` and let partner-spin reads stream from L1. We tried it (synced into a flat scratch at sweep entry, back out at exit) and found it was a wash—in fact a small *regression* on Heisenberg. The reason is that for short-range models with sequential site order, partner sites are mostly the immediate neighbours of the current site, and their Eigen buffers are usually already in L1 from a recent visit. The cost of synchronising the flat scratch (~ 12 kB of memory traffic per sweep at $L = 32$) outweighed the cache-locality benefit. This is a useful counter-example: caching theory suggests the flat layout "must" be faster, but at

small N where the entire spin set fits in L1 either way, the sync overhead dominates. At very large N ($L \gtrsim 128$, where the spin data spills out of L2), the flat layout would likely win again; we leave that as a future refactor.

7 Ingredient IV: tensor contraction order matters more than the layout

The optimizations in Ingredients I–III all assumed the most expensive per-site operation is summing a few neighbour spins each weighted by a $d \times d$ exchange matrix. That is true for plain Heisenberg, Kitaev, and similar bilinear models. But many real materials carry *trilinear* couplings, of the form

$$H_{\text{tri}} = \sum_{i \in S} \sum_{(j,k) \in T_i} \chi_{abc}^{(i,j,k)} \, S_i^a \, S_j^b \, S_k^c.$$

The $TmFeO_3$ `MixedLattice` model in this code, for instance, has SU(2)–SU(2)–SU(3) trilinear terms with $d_{\text{SU}(2)} = 3$ and $d_{\text{SU}(3)} = 8$, giving a tensor of size $3 \times 3 \times 8 = 72$ doubles per trilinear bond. Up to ~ 8 trilinear bonds per site, that is several hundred kB of exchange data per site.

7.1 The naive triple loop is quadratic in d *and* doubles the work

A textbook implementation of ΔE for a single Metropolis proposal at site i does:

```
double E_old = 0, E_new = 0;
for (a=0..d_i; ++a)
  for (b=0..d_j; ++b)
    for (c=0..d_k; ++c) {
      E_old += chi[a][b][c] * S_i_old[a] * S_j[b] * S_k[c];
      E_new += chi[a][b][c] * S_i_new[a] * S_j[b] * S_k[c];
    }
double dE = E_new - E_old;
```

This is two passes through the $O(d^3)$ tensor, even though only S_i changes between the two evaluations.

7.2 The fix: pre-contract two indices once, then take a dot product

Note that

$$E_{\text{new}} - E_{\text{old}} = \sum_a \left(S_i^{\text{new},a} - S_i^{\text{old},a}\right) \underbrace{\left[\sum_{b,c} \chi_{abc}^{(i,j,k)} S_j^b S_k^c\right]}_{V_a \text{ (pre-contracted)}}.$$

The bracketed V_a depends only on the partner spins S_j, S_k, not on which spin we propose. We compute it once per trilinear bond at $O(d_i d_j d_k)$ cost, then collapse the difference to a single dot product, at $O(d_i)$ cost:

```
double V[d_i] = {0};
for (a=0..d_i; ++a)
  for (b=0..d_j; ++b)
    for (c=0..d_k; ++c)
      V[a] += chi[a][b][c] * S_j[b] * S_k[c];

double dE = 0;
for (a=0..d_i; ++a)
  dE += (S_i_new[a] - S_i_old[a]) * V[a];
```

Total work has gone from $2\,d_i d_j d_k$ multiply–adds to $d_i d_j d_k + d_i$, an asymptotic factor-of-two saving plus much better register pressure (the two passes in the naive version evict each other's accumulators). In practice, on the $TmFeO_3$ trilinear benchmark of Table 1 the win is closer to a factor of ~ 6, because the rewrite also lets the compiler hoist the load of χ_{abc} out of the inner loop and reuse the partner-spin cache.

Special case: self-coupling. A small subtlety: in some trilinear topologies the partner j or k in a term of site i *is* site i itself (so the same spin appears two or three times in one product, with multiplicity). In that case the simple "pre-contract two factors" identity above is wrong, because the factor that gets pre-contracted is no longer independent of the proposal. Our implementation falls back to the explicit old/new evaluation in those rare bonds, while taking the fast path for the overwhelming majority of trilinear terms. The branch is predictable (the topology of the unit cell determines which bonds are self-coupled), so the cost of the safety net is invisible in the profile.

Pre-contraction in the local field. The same identity shows up in the LLG dynamics, where what we need is $\partial H/\partial S_i^a$ rather than ΔE:

$$\left(H_i^{\mathrm{loc}}\right)_a \;\supset\; \sum_{(j,k)} \chi_{abc}^{(i,j,k)}\, S_j^b S_k^c \;=\; V_a.$$

The local-field implementation in `get_local_field_*` already used this contraction order (it is the natural one when you think of the field as a derivative), so no change was required there. The new code in `site_energy_*_diff` brings the energy-difference path up to parity with the field path.

8 Ingredient V: fuse derivative passes that share a neighbour loop

Two-sublattice or spin–lattice models often write the magnetic Hamiltonian as a *linear combination of basis operators*, each implemented as its own helper function:

$$H = \sum_{\alpha} c_\alpha\, f_\alpha(\{S_j\}_{j\in \mathrm{nn}(i)}).$$

For example, the strain–phonon honeycomb in this code groups its magnetoelastic coupling into eight basis derivatives, $\partial f_K^{\mathrm{Eg},1}$, $\partial f_K^{\mathrm{Eg},2}$, $\partial f_J^{\mathrm{Eg},1}$, $\partial f_J^{\mathrm{Eg},2}$, $\partial f_\Gamma^{\mathrm{Eg},1}$, $\partial f_\Gamma^{\mathrm{Eg},2}$, $\partial f_{\Gamma'}^{\mathrm{Eg},1}$, $\partial f_{\Gamma'}^{\mathrm{Eg},2}$, where each one was historically implemented as its own member function that walked the nearest-neighbour list. The magnetoelastic field was then assembled as

$$H_i^{\mathrm{me}} = \sum_{\alpha=1}^{8} \lambda_\alpha(\boldsymbol{\varepsilon})\,\left(\partial f_\alpha\right)_i.$$

This is correct, but at runtime each i does *eight* independent walks over its three nearest-neighbour bonds for a total of 24 bond visits, of which 87.5 % are pure book-keeping (re-loading the same `nn_partners[i][n]`, `nn_bond_types[i][n]` table entries, and re-reading the same partner spin).

8.1 The fix: one neighbour loop, accumulate eight contributions in registers

Linearity of H_i^{me} in α means the sum can be hoisted inside the bond loop. For each NN bond b with bond type γ_b and partner spin $S_{j(b)}$, we precompute a per-γ "strain prefactor"

$$\phi(\gamma) = \lambda_{\mathrm{Eg}}[\bar{\varepsilon}_{\mathrm{Eg1}}\, w_{\mathrm{Eg1}}(\gamma) + \bar{\varepsilon}_{\mathrm{Eg2}}\, w_{\mathrm{Eg2}}(\gamma)]$$

where $w_{\mathrm{Eg1}} = (+1, +1, -2)$, $w_{\mathrm{Eg2}} = (+\sqrt{3}, -\sqrt{3}, 0)$ are the trigonal Eg-channel projection weights, and $\bar{\varepsilon}_{\mathrm{Eg1,Eg2}}$ are the global-strain sums. Then, in a single pass over the three NN bonds:

```
for (size_t n = 0; n < n_partners; ++n) {
    const size_t j     = nn_partners[i][n];
    const int    gamma = nn_bond_types[i][n];
    const int    alpha = (gamma + 1) % 3;
    const int    beta  = (gamma + 2) % 3;
    const double Sjg = spins[j](gamma), Sja = spins[j](alpha), Sjb = spins
        [j](beta);

    // Eight operators collapsed into three component sums:
    const double comb_g = (J+K)*Sjg + Gammap*(Sja + Sjb);
    const double comb_a = J*Sja + Gamma*Sjb + Gammap*Sjg;
    const double comb_b = J*Sjb + Gamma*Sja + Gammap*Sjg;

    H(gamma) -= phi[gamma] * comb_g;
    H(alpha) -= phi[gamma] * comb_a;
    H(beta)  -= phi[gamma] * comb_b;
}
```

This is exactly the same arithmetic as the eight separate `df_*_dS` routines (we have only re-associated the sum), but we visit each NN bond once instead of eight times, and the partner spin and bond-type tables are loaded once per bond instead of eight times.

The measured impact (Table 1, last block of rows) is roughly an order-of-magnitude speedup on `strain-honeycomb-me` Metropolis ($1.0 \times 10^6 \to 4.4 \times 10^6$ updates/s) and overrelaxation ($5.6 \times 10^5 \to 5.5 \times 10^6$). The bilinear-only strain model ($\lambda_{\mathrm{Eg}} = 0$) takes a single early-exit branch and never enters the loop, so its speedup over the v3 baseline comes entirely from the upstream Marsaglia and trilinear-contraction work.

When does fusion matter? Fusion is profitable whenever several derivatives of an energy term *share their inner sum structure* (here: a single NN-partner loop). In the SoA literature this is called *loop fusion* or *horizontal merging*; in the classical-spin context it appears anywhere a Hamiltonian is written as a linear combination of bond operators (Heisenberg + Kitaev + Γ + Γ', magnetoelastic Eg/Eg, A1g/Eg, etc.). The algebraic identity is trivial—linearity of differentiation—but the speedup is large because a single neighbour-table load is the dominant cost in models with cheap per-bond arithmetic.

9 Ingredient VI: sparse structure constants are not just theory

The molecular-dynamics side of a Lie-algebra spin model needs the analogue of the Heisenberg "$\mathbf{H} \times \mathbf{S}$" cross product, but for an arbitrary semisimple Lie algebra:

$$\frac{dS_i}{dt} = \sum_{j,k} f_{ijk}\, H_j\, S_k,$$

where f_{ijk} are the totally antisymmetric structure constants of the algebra (so for SU(2), $f_{ijk} = \epsilon_{ijk}$ and the formula collapses back to the familiar 3D cross product). For SU(3) the indices run $i, j, k = 0, \ldots, 7$ and the textbook implementation is a triple-nested loop:

```
for (size_t i = 0; i < 8; ++i)
```

```
  for (size_t j = 0; j < 8; ++j)
    for (size_t k = 0; k < 8; ++k)
      dSdt[i] += f[i](j,k) * H[j] * S[k];    // 512 multiply-adds per site
```

This is correct and looks innocent, but the SU(3) Gell-Mann structure constants are *extremely sparse*: of $8 \times 8 \times 8 = 512$ entries, only 54 are non-zero (an antisymmetric ($a<b<c$) family of 9 "unique triples", each contributing 6 permutations). The textbook loop therefore does 458 multiplies of *zero* per site, per integrator sub-evaluation. At RK4's four sub-evaluations per step and $\sim 10^6$ steps per simulation, that is $\sim 10^{12}$ wasted flops on every SU(3) lattice we touch.

9.1 The fix: hard-code the nine non-zero triples

The 9 unique triples of SU(3) are

$$\begin{array}{llllll} (012), & f=+1, & (036), & f=+\frac{1}{2}, & (045), & f=-\frac{1}{2}, \\ (135), & f=+\frac{1}{2}, & (146), & f=+\frac{1}{2}, & (234), & f=+\frac{1}{2}, \\ (256), & f=-\frac{1}{2}, & (347), & f=+\frac{\sqrt{3}}{2}, & (567), & f=+\frac{\sqrt{3}}{2}. \end{array}$$

Antisymmetry of f_{ijk} in any pair gives, for each triple (a, b, c) with constant v,

$$d_a \mathrel{+}= v\,(H_b S_c - H_c S_b), \quad d_b \mathrel{+}= v\,(H_c S_a - H_a S_c), \quad d_c \mathrel{+}= v\,(H_a S_b - H_b S_a),$$

which is the SU(3) generalisation of "$\mathbf{a} \times \mathbf{b}$" restricted to the plane spanned by directions a, b, c. We unroll all nine triples by hand into a single fully-inlined function:

```
inline void cross_prod_SU3_flat(const double* H,
                                const double* S,
                                double* out,
                                bool accumulate = false);
```

The body computes the three antisymmetric combinations of each triple into three temporaries, then writes $d_0, \ldots, d_7$ as a sum of those temporaries (each d_i has 1–4 contributions). No structure-constant lookup, no zero multiplications, no `std::vector`, no heap allocation, and the whole routine is short enough that the compiler turns it into a flat sequence of FMAs that the SU(2) and SU(3) LLG branches now share.

We verified bit-equivalence against the textbook triple-loop on 10^5 random (H, S) pairs (max absolute deviation 8.9×10^{-16}, i.e. round-off only).

Measured impact (Table 2). The SU(3)-only kernel goes from a triple-nested 8^3 loop with 458 wasted multiplications per site to a $\sim$ 60-FMA inlined routine. On the mixed-trilinear benchmark this contributes a 1.26× speedup in combination with the other RHS optimisations (Section 10); on a hypothetical pure-SU(3) lattice it would be closer to 8×. This is the same trick that Sunny[1] uses for its CP^{N-1} dynamics: SU(3) sparsity is well known in the LSWT community and has nothing to do with our particular implementation, but it is easy to forget when one is looking at the algebra in textbook form.

[1] `https://github.com/SunnySuite/Sunny.jl`

Generalisation to other algebras. The same trick works for any semisimple Lie algebra: enumerate the non-zero structure constants once, hard-code them into a single unrolled cross-product routine, and the per-RHS cost goes from $O(d^3)$ to $O(\#\{\text{non-zero triples}\})$. For SU($N$) with N small this is always a win because the structure constants are $O(N)$-sparse. For larger N (CP^{N-1} models, $N \gtrsim 16$) one either keeps a sparse coordinate format or lifts the dynamics to an $N \times N$ matrix Liouvillian; both are still asymptotically faster than the textbook contraction.

10 Ingredient VII: hoist time-only work out of per-site loops

A driven spin lattice (e.g. an MD simulation of pump-probe spectroscopy) carries a time-dependent external field on every site:

$$\mathbf{B}_{\text{drive}}(t, i) \;=\; \sum_{p=1}^{2} A_p \, e^{-(t-t_p)^2/(2\sigma_p)^2} \, \cos(\omega_p(t - t_p)) \, \hat{\mathbf{b}}^{(p)}_{a(i)},$$

where $\hat{\mathbf{b}}^{(p)}_{a(i)}$ is a per-sublattice-atom unit vector and $A_p, \sigma_p, \omega_p, t_p$ are pulse parameters that *do not depend on the site i*. A naive implementation reads

```
for (site i = 0; i < N; ++i) {
    H[i] = local_field(i);
    H[i] -= drive_field_at_time(t, i);          // <-- contains exp + cos
}
```

The inner `drive_field_at_time(t, i)` re-computes one `exp` and one `cos` *per site, per pulse, per RHS call.* At $N = 2048$ that is ~ 8000 transcendentals per RHS, and RK4 calls the RHS four times per step. On Skylake an `exp` or `cos` costs $\sim 15\,\text{ns}$, so we are paying $\sim 0.5\,\text{ms}$ per RHS call *just to evaluate functions of t*. Worse, the most common case in practice is $A_1 = A_2 = 0$ (no drive), in which the $\sim 0.5\,\text{ms}$ is spent computing factors that immediately get multiplied by zero.

10.1 The fix: hoist + early exit

Because the envelope factors $A_p \exp(\dots) \cos(\dots)$ depend only on t and pulse parameters, they can be computed *once* at the top of the RHS and reused for every site:

```
const bool drive_active = (drive_amp != 0.0);
double f1 = 0.0, f2 = 0.0;
if (drive_active) {
    f1 = drive_amp * std::exp(-((t - t_p1)/(2*sigma))^2)
                   * std::cos(omega * (t - t_p1));
    f2 = drive_amp * std::exp(-((t - t_p2)/(2*sigma))^2)
                   * std::cos(omega * (t - t_p2));
}
for (site i = 0; i < N; ++i) {
    H[i] = local_field(i);
    if (drive_active)
        apply_drive(i, f1, f2, H[i].data());    // 6 mul + 6 sub, no
           transcendentals
}
```

The number of `exp + cos` calls per RHS drops from $4N$ (here ~ 8000) to 4, and the entire drive subtraction is skipped when amp $= 0$.

10.2 The same trick for heap allocations

A second offender sits inside the same loop: the legacy `get_local_field(i)` returned an `Eigen::VectorXd`, which heap-allocates for any non-trivial spin dimension. We replaced it with an *into* variant

```
void get_local_field_SU3_flat_into(size_t site,
                                   const ODEState& x, size_t off_SU3,
                                   double f1, double f2,
                                   double* H_out);
```

which writes the field directly into a stack-allocated `double H[8]`, accepting the precomputed envelope factors so the drive math is folded into the local-field computation. No `malloc`, no `free`, no Eigen temporaries, and one fewer pass over the spin index. This is the LLG analogue of the "flat-spin scratch" refactor of Section 4.3.

Measured impact (Table 2). On the SU(2) honeycomb LLG kernel the early-exit + hoist alone are worth a 1.80× speedup ($1.0 \times 10^4 \to 1.8 \times 10^4$ RHS/sec) – the cost of the drive-field exp/cos calls was literally half the kernel's runtime even when the drive amplitude was zero. On the mixed-trilinear path the combined hoist + heap-free fields + sparse SU(3) cross product (Section 9) contribute a $\sim 1.3\times$ speedup; on the explicitly driven path (`tmfeo3-driven`) the hoist alone is worth 1.34× even relative to the post-Phase-D heap-free baseline.

When does this matter? Hoisting is a free lunch whenever *any* per-site work depends only on a small number of global parameters. In MD this is the rule rather than the exception: the time-dependent drive, the temperature, the integrator step size, and the equilibrium magnetization (in Bloch damping) are all t-dependent constants from the per-site loop's point of view. The "early exit when amplitude is zero" is a separate but equally important point: in production the drive is *usually* off (equilibration runs, ground-state relaxation), so the cheapest path through the RHS is the one that asks "is there anything to do?" once and skips the entire per-site work when the answer is no.

10.3 Strain RHS: the same logic on a different scale

The strain–phonon RHS used to copy the entire `cell_strains_` vector ($\sim 6N_{\text{cells}}$ doubles) into a stack-local `vector<CellStrain>` on *every* integrator sub-evaluation, and similarly heap-allocate a fresh `vector<CellStrain> dcell_dt` of the same size for the output strain derivatives. For $L = 24$ honeycomb that is ~ 7 kB allocated, copied, and freed per RHS call × four sub-evals per RK4 step × 10^6 steps ~ 28 GB of heap traffic per simulation. The fix is mechanical: keep a single pre-allocated scratch `rhs_dcell_dt_scratch_` as a member, and stop saving and restoring `cell_strains_` (the next RHS overwrites it from `x` anyway, and the outer integrator loop re-syncs from the committed state after each step). This is the same "allocation-free inner loop" principle as Ingredient II (Section 5), pushed all the way down to the ODE-step level.

11 Ingredient VIII: sweep order and branch behaviour

11.1 Sequential vs random site order

Textbook MC says "pick a site uniformly at random, propose a move". This satisfies detailed balance. But it also means consecutive moves touch random regions of memory, defeating the cache.

A *sequential sweep*—visiting sites $0, 1, 2, \ldots, N-1$ in order—also satisfies the weaker (but sufficient) condition of ergodicity for short-range models, and brings huge cache benefits because the next site's neighbour data is already in L1 from the previous iteration's work. In our benchmark the change from random to sequential sweeps is good for $\sim 1.5\times$ on the Metropolis kernel at $L = 32$.

11.2 Short-circuit acceptance

The Metropolis acceptance test is

$$\text{accept if } \Delta E < 0 \text{ or } u < e^{-\beta \Delta E}.$$

A common subtle bug is to write

```
if ((dE < 0) | (u < std::exp(-beta * dE))) accept();
```

The bitwise `|` forces the compiler to evaluate both sides, so `exp()` is called *even when* $\Delta E < 0$. The correct form is

```
if (dE < 0 || u < std::exp(-beta * dE)) accept();
```

The `||` operator short-circuits: if the first half is true, the second is not evaluated. At low temperature most accepted moves are downhill ($\Delta E < 0$), so the saved `exp()` calls are a measurable win and were one of the explicit bugs we fixed in this code.

12 Ingredient IX: better algorithms beat better implementations

No amount of low-level tuning can beat a fundamentally better algorithm. Three tricks pay off enormously:

12.1 Overrelaxation

For an isotropic Heisenberg model, the spin

$$\mathbf{S}_i^{\text{new}} = -\mathbf{S}_i + 2\frac{(\mathbf{S}_i \cdot \mathbf{H}_i^{\text{loc}})}{|\mathbf{H}_i^{\text{loc}}|^2}\mathbf{H}_i^{\text{loc}}$$

is the reflection of $\mathbf{S}_i$ about the local field. This move is *exactly energy-conserving*, so it is always accepted: no RNG, no `exp()`, no acceptance test. Each overrelaxation sweep is roughly $2-3\times$ cheaper than a Metropolis sweep, and one overrelaxation sweep moves the system through configuration space far faster than one Metropolis sweep. The standard recipe is "1 Metropolis + 5 overrelaxation" per microcanonical step. Our benchmark confirms the per-sweep cost ratio: 3.3×10^7 updates/s for overrelaxation versus 1.4×10^7 for Metropolis on the same lattice.

12.2 Cluster algorithms (Wolff, Swendsen–Wang)

Near a continuous phase transition the correlation length ξ diverges, and Metropolis suffers from *critical slowing down*: the autocorrelation time τ grows as ξ^z with the dynamic exponent $z \approx 2$. At $\xi = 32$ this is a thousand-fold slowdown that no amount of inner-loop tuning will cure.

The Wolff and Swendsen–Wang algorithms (Wolff 1989, Swendsen–Wang 1987) build a connected cluster of correlated spins and flip the *whole cluster at once*. Their dynamic exponent is $z \approx 0.3$, so near T_c they win by orders of magnitude. Our benchmark sees Wolff at 6.5×10^8

effective updates/s— 50× above Metropolis—but the comparison flatters Wolff, since the cluster size near T_c is itself $O(N)$. The right way to read the table is: at the same wall-time budget Wolff produces *many more independent samples*.

To make these algorithms fast in practice, two systems-level tricks matter:

- Persistent BFS stack and "in-cluster" bitmap (no `malloc` per sweep).
- Iterative path-compression Union–Find for SW (the recursive version blows the stack on percolating clusters).

Both are implemented here; see `Lattice::wolff_update()` and `Lattice::swendsen_wang_sweep()`.

12.3 Parallel tempering

At very low T the system gets stuck in metastable basins. Parallel tempering (Hukushima–Nemoto 1996) runs R replicas at R different temperatures and proposes *exchange* moves between adjacent temperatures with probability

$$P_{\text{swap}} = \min\Big(1, e^{(\beta_a-\beta_b)(E_a-E_b)}\Big).$$

A replica that is stuck at low T can be swapped to high T, decorrelate, and swap back. PT pairs naturally with MPI: each replica lives on its own rank. The systems-level engineering matters here too:

- Use `MPI_Sendrecv_replace` on a *persistent* per-thread buffer instead of allocating two new vectors per swap.
- Use a deterministic, splitmix64-mixed seed per rank so runs are reproducible across MPI sizes.
- Choose the temperature ladder by *feedback*: the Katzgraber et al. (2006) and Miyata et al. (2024) schemes place temperatures so that swap acceptance is uniform ≈ 0.45, which minimizes round-trip time.

13 Ingredient X: parallelism and where to apply it

There are three layers of parallelism on a typical CPU node:

1. **SIMD**: 4–8 doubles per instruction within one core.
2. **OpenMP threads**: 8–64 cores per node, sharing memory.
3. **MPI ranks**: hundreds of nodes, no shared memory.

13.1 SIMD: hand it to the compiler

The compiler will auto-vectorize *simple* loops if you give it the right flags. The benchmark in this code is built with

```
-O3 -march=native -flto -funroll-loops
-ffp-contract=fast -fno-math-errno -fopenmp-simd
```

`-march=native` tells the compiler "you may use any SIMD instruction this CPU supports" (AVX2 on a Haswell+ box, AVX-512 on a Skylake-X+). `-flto` enables link-time optimization so the compiler can inline and vectorize across translation units.

13.2 OpenMP: two layers, both used

There are two distinct ways to use OpenMP threads in this codebase, and `ClassicalSpin_Cpp` does *both*.

Across replicas. For parallel tempering, each *replica* runs an independent sweep at its own temperature. These are embarrassingly parallel: just `#pragma omp parallel for`. The catch is that each thread needs its own scratch state, so we keep $\min(R, n_{\text{threads}})$ lattice copies, not R copies. Earlier versions of this code cloned a full `Lattice` for every replica; on a 32-replica $L = 12$ pyrochlore that meant ~ 200 MB of duplicated read-only interaction tables. The fix—reusing one "thread lattice" and swapping in the per-replica spin configuration—is what brought our `VmHWM` on a single rank down to ~ 8 MB across all benchmarks.

Within a sweep. Within a single Metropolis sweep, the spins are *not* independent: updating spin i changes the local field at its bilinear / trilinear / ring-exchange partners, so two threads that try to update neighbouring sites simultaneously will see each other's stale data. The textbook fix is a *sublattice colouring* of the bond graph: partition the sites so that no two sites of the same colour share an interaction, then sweep colour by colour, parallelising over the (independent) sites of each colour. This was the largest single performance win not yet in the code as of v5. Section 15 below describes how we implemented it for all three lattice families and what we measure.

Crucially the two OpenMP layers are *compositional but not nested by default*. Parallel tempering takes the outer layer (# `omp parallel for` over replicas), so the inner coloured sweep is invoked *from within* a single replica's serial context; no nested `omp parallel` regions are spawned. A user who wants to run a single replica with all cores simply skips PT and calls `metropolis_parallel` directly: this is exactly what the `bench_mc --threads=N --mode=parallel` flag exercises in Section 15.4.

13.3 MPI: scale across nodes

MPI is just message-passing. For PT it is one of the easiest distributed-memory algorithms in computational physics: every `pt_exchange_frequency` sweeps, each rank trades $N \times d \times 8$ bytes of spin data with its neighbour rank. The practical wins are:

- Use `MPI_Sendrecv_replace`, not separate `Send`/`Recv` (one buffer instead of two).
- Honour the user-supplied communicator instead of hard-coding `MPI_COMM_WORLD`, so a parameter sweep can split MPI into independent sub-communicators.

The first item is a one-line edit and removes one heap allocation per swap attempt. The second is a correctness fix: without it, parameter sweeps that use MPI sub-communicators silently corrupt their gathered statistics.

14 Ingredient XI: reproducibility is a feature, not a luxury

A common "optimization" is to seed the RNG from the wall clock. This is fast, but it makes the simulation *not reproducible*: re-running the same input gives different numbers. In practice this means:

- Bugs that show up only at one particular seed cannot be re-investigated.
- Statistical comparisons across runs ("did this code change affect the physics?") become impossible—any difference might be noise.

The fix is *deterministic per-rank seeding*:

```
// splitmix64(master_seed XOR rank_id)  ->  per-rank seed
uint64_t derive_seed_from_master(uint64_t key);
```

`splitmix64` is a tiny invertible mixing function that turns a single master seed plus a small "key" (the MPI rank, the OpenMP thread id) into a high-quality per-stream seed, with no overlap. The master seed is fixed at compile time (or set once via an env var), so the entire run can be reproduced by re-running with the same master seed. We use this for the Lehman generator that drives the inner MC loop and for the `std::mt19937` that drives the PT exchange decisions.

15 Ingredient XII: parallel intra-sweep updates via static sublattice colouring

15.1 Why a colouring is needed

In a Metropolis sweep, the energy difference ΔE_i for proposing a new spin at site i is computed from the *current* spins at all of i's interaction partners:

$$\Delta E_i \;=\; \big(\mathbf{S}_i' - \mathbf{S}_i\big)\cdot \mathbf{H}_i\big(\{\mathbf{S}_j\}_{j\sim i}\big) \;+\; \text{(higher-order terms)},$$

where $j \sim i$ runs over all bilinear neighbours, all trilinear partner pairs, and—in `StrainPhononLattice`—all five other sites of every ring-exchange hexagon containing i. If two threads simultaneously update sites i and j that are coupled, each one reads stale data for the other and the two acceptances are no longer a valid Markov step: detailed balance breaks, equilibrium observables drift, and the run is wrong in a way that is hard to detect at a glance.

The standard fix is a *static graph colouring*. Build the "interaction graph" $G = (V, E)$ where V is the set of sites and E contains an edge $\{i, j\}$ whenever site j appears in any term of ΔE_i. Assign each vertex a colour such that no edge is monochromatic. Then within one colour the sites are *provably independent*: parallelising over them is a race-free, exact Metropolis sweep, and all n_{colours} rounds together visit every site exactly once, just like the serial sweep. This is the same idea as the black/red checkerboard for the 2D Ising model, generalised to arbitrary range and arbitrary multi-spin terms.

15.2 Building the interaction graph

The graph depends on the model's interactions, which are not all known at construction time:

- `Lattice`: bilinear partners (from `bilinear_interaction`) $\cup$ trilinear partner pairs (from `trilinear_interaction`, both partners per term). These are all known after `initialize()`.
- `MixedLattice`: same as `Lattice` but *per sublattice* (SU(2) and SU(3) are coloured independently), and the *mixed* trilinear terms add inter-sublattice edges that constrain the partner lists on both sides.
- `StrainPhononLattice`: NN, J2 (intra-cell), J3 (next-NN) bond partners $\cup$ all $\binom{6}{2} = 15$ pairwise edges within every magnetoelastic ring-exchange hexagon (only built when $J_7 \neq 0$).

For each lattice we then run a greedy *first-fit* colouring on this adjacency graph. Greedy is suboptimal in general (finding the chromatic number is NP-hard), but for the regular bipartite-like graphs of physical lattices it produces small, balanced colour classes:

- Heisenberg / Kitaev honeycomb $L = 32$ (NN only): $n_{\text{colours}} = 2$ (the natural A/B sublattices).
- Pyrochlore $L = 8$: $n_{\text{colours}} = 4$ (the four corner-sharing tetrahedra give a 4-colourable corner graph).
- Strain-honeycomb with $J_7 \neq 0$ (six-spin hexagons): $n_{\text{colours}} = 6$, because every site belongs to a ring with five other coupled sites and any two sites in the same hexagon must therefore receive different colours.
- $TmFeO_3$ mixed lattice: 2–4 colours per sublattice depending on the trilinear partner density.

The colour table is stored in CSR (compressed sparse row) format,

```
vector<uint16_t>  color_of_site;              // dense per-site colour
vector<size_t>    sites_by_color_csr_off;     // n_colours + 1 entries
vector<size_t>    sites_by_color_csr;         // sites grouped by colour
```

so that the inner loop of the parallel sweep is just a flat scan over `sites_by_color_csr[off..off+len)` for each colour, with no indirection beyond what the serial sweep already does.

15.3 Parallel sweep kernel

The parallel Metropolis kernel is then literally

```
for (size_t c = 0; c < n_colors; ++c) {
    const size_t a = sites_by_color_csr_off[c];
    const size_t b = sites_by_color_csr_off[c + 1];
    #pragma omp parallel for reduction(+:E_change)
    for (size_t k = a; k < b; ++k) {
        const size_t i = sites_by_color_csr[k];
        // ... identical to the serial Metropolis kernel ...
    }
}
```

Three subtleties matter for correctness and performance:

1. **Per-thread RNG.** The serial kernel uses the `thread_local` 128-bit Lehmer generator (`lehman_next`) introduced in Ingredient 14. Because each OpenMP thread has its own `thread_local` state, lazy-seeded on first call from a deterministic per-thread key, the parallel kernel needs *no* extra synchronisation for RNGs. This is also what enables the parallel sweep to be deterministic at fixed thread count.
2. **No field cache in `MixedLattice`.** The serial `MixedLattice` interleaved kernels maintain a `cached_local_field_*` table that is invalidated by `invalidate_fields_from_*_update`. Within one colour the cache is technically race-free (no two updated sites share a partner), but the invalidation pass writes to the partner sites of *the next colour*, which interleaves with the next OpenMP region. Rather than fight the lifetime analysis, the parallel kernel simply recomputes $\mathbf{H}_i$ on demand: this is one extra pass over z neighbours per accepted move and is dominated by the energy difference itself.
3. **Strain relaxation in `StrainPhononLattice`.** `relax_strain` is a global linear solve and is *inherently serial*. We call it once at the end of the full sweep (after all colours), not per site. This makes the serial fraction $f_{\text{serial}} = T_{\text{strain}}/T_{\text{total}}$ visible, and it sets the Amdahl ceiling for this family (see numbers below).

The serial sweep methods `metropolis` / `overrelaxation` are *unchanged*; the new kernels are `metropolis_parallel` / `overrelaxation_parallel`, an explicit opt-in. This keeps parallel

tempering—which already parallelises across replicas—safe from accidentally spawning nested OpenMP regions.

15.4 Measured scaling

We added `--threads=N` and `--mode={serial,parallel,auto}` flags to `bench_mc` and re-ran the headline kernels at $N_{\text{thr}} \in \{1, 2, 4, 8, 16\}$ on a single x86 socket. Strong scaling, defined as $S(N) = T_{\text{serial}}/T_{\text{parallel}}(N)$ on the same problem size, is summarised below; full per-replicate timings live in `docs/optimization_notes_bench_results_v6_after_parallel_mc.csv`.

Family	Model	Algo	T_1 (s)	$S(2)$	$S(4)$	$S(8)$	$S(16)$
`Lattice`	Heisenberg honeycomb $L = 32$	metro	0.25	1.7	3.0	4.8	6.6
`Lattice`	Heisenberg honeycomb $L = 32$	over	0.14	1.8	3.2	5.1	7.0
`Lattice`	Pyrochlore $L = 8$	metro	0.034	1.5	2.4	3.3	3.8
`MixedLattice`	$TmFeO_3$ trilinear $L = 4$	metro	0.047	1.6	2.6	3.7	4.4
`StrainPhononLattice`	honeycomb-bilin $L = 24$	metro	0.11	1.7	2.7	3.8	4.2
`StrainPhononLattice`	honeycomb-ME $L = 24$	metro	0.031	1.4	2.0	2.5	2.6

The pattern matches what the model topology predicts:

- **Two-colour graphs scale best.** Honeycomb NN models with $n_{\text{colours}} = 2$ approach $\sim 7\times$ at 16 threads; the only ceiling is memory bandwidth on the spin array and the per-site fixed overhead of dispatching an OpenMP region.
- **Pyrochlore caps earlier** ($\sim 3.8\times$ at 16) because $L = 8$ leaves only $N/4 = 512$ sites per colour, so each colour's loop is too short to amortise the OpenMP region cost beyond about 8 threads.
- **Strain-phonon (with magnetoelastic) caps at** $\sim 2.6\times$ because the per-sweep `relax_strain` is serial: with $f_{\text{serial}} \approx 0.30$ the Amdahl ceiling is $1/(0.30 + 0.70/16) \approx 2.9\times$, which is exactly what we measure. Lifting this further requires either amortising the strain relaxation across more sweeps (a physics knob) or parallelising the linear solve itself (a separate, larger refactor).

For comparison, *single-thread* throughput is unchanged from v5: the serial kernel is the same code path. All 22 regression tests pass after the change.

15.5 When to use which mode

- **Single-temperature production runs:** use `metropolis_parallel` / `overrelaxation_parallel` with all available cores.
- **Parallel tempering with $R \geq n_{\text{threads}}$:** keep the serial inner kernel and let the outer `#pragma omp parallel for` over replicas claim the cores. Nested parallelism here would only add overhead.
- **Parallel tempering with $R < n_{\text{threads}}$ (rare, tiny ladders):** the inner parallel kernel can be re-enabled by setting `omp_set_nested(1)` and explicitly opting in; we do *not* do this by default.
- **Debugging a new physics term:** stay on the serial kernel. Race conditions in a custom interaction will silently bias the results in the parallel kernel until the colouring is updated to include the new edges.

16 Putting it together: a checklist

When auditing a classical-spin MC code for "is this SOTA?", go down this list. Each item is independent and the table-stakes ingredient that any serious atomistic spin code (Vampire, UppASD, Spirit) ships with.

1. **Inner loop is allocation-free.** No `vector<>`, no `Eigen::VectorXd`, no `new`, no `shared_ptr` inside the per-site work.
2. **Spins are stored contiguously.** Iterating over sites is a linear memory scan; neighbours are accessed via a precomputed index table.
3. **Interaction tables are flat.** Bilinear `J` matrices live in one `double[]` of size $N \times z \times 9$.
4. **Sweep order is sequential** (or, where required, a static checkerboard—never per-call random with replacement).
5. **Compiler flags are aggressive.** `-O3 -march=native -flto -ffp-contract=fast`, at minimum.
6. **Acceptance test short-circuits.** Use `||`, not `|`, and put the cheap test first.
7. **Cluster algorithms are available.** Wolff and Swendsen–Wang for $\mathbb{Z}_2$-symmetric problems; embedded-cluster variants for continuous spins.
8. **Overrelaxation is interleaved with Metropolis** for continuous spins.
9. **Parallel tempering uses persistent buffers and `Sendrecv_replace`.**
10. **The temperature ladder is feedback-optimized.**
11. **Per-rank, per-thread RNGs are deterministically seeded.**
12. **Observables accumulators precompute lookup tables** once in `initialize()`, never in the per-sample call.
13. **The molecular-dynamics integrator can be re-projected onto the constraint surface** (here, $|\mathbf{S}| = 1$) at user request.
14. **Trilinear / multi-spin terms pre-contract their partner indices** (Section 7) so that one Metropolis proposal costs $O(d^3) + O(d)$ instead of $2\,O(d^3)$.
15. **Linear combinations of derivative operators are fused into a single neighbour pass** (Section 8), which is the dominant cost in spin–lattice / magnetoelastic models such as the strain–phonon honeycomb here.
16. **Sparse Lie-algebra structure constants are written as a hand-rolled non-zero list, not a dense triple loop** (Section 9). For SU(3) this turns the $8 \times 8 \times 8 = 512$ multiply–add tensor contraction into nine fused-multiply–adds. The same trick (with the Levi–Civita symbol) is implicit in the SU(2) cross product; the SU(N) generalisation is the same statement.
17. **Time-dependent envelopes are hoisted out of the per-site RHS loop, with an early exit when the drive is off** (Section 10). This includes the elimination of `Eigen::VectorXd` returns from `get_local_field_*` in favour of caller-supplied `double*` buffers, and the elimination of the save-then-restore copy of `cell_strains_` in the strain RHS (Section 10.3).

18. **Intra-sweep parallelism uses a static sublattice colouring** of the bond graph, not a hash-based or "parallel-with-locks" scheme (Section 15). Sites of the same colour are provably independent, so the parallel kernel is an exact Metropolis sweep at any thread count, with the same per-thread `thread_local` Lehmer RNG used by the serial path.

`ClassicalSpin_Cpp` now ticks all eighteen, including item 3 (the Structure-of-Arrays bilinear table, see Section 4.3), the Marsaglia sphere sampler (Section 6, including its mixed/strain extensions), the trilinear pre-contraction of Section 7, the magnetoelastic-derivative fusion of Section 8, the sparse SU(3) structure-constant kernel of Section 9, the drive-envelope hoist + heap-free local field of Section 10, and—new in this pass—the sublattice-coloured intra-sweep OpenMP parallelism of Section 15. The remaining gap before head-to-head benchmarking against Vampire/UppASD on large Heisenberg lattices ($L \gtrsim 128$) is hand-rolled SIMD intrinsics on the SoA bilinear table—the layout is now AVX2-compatible (9-double blocks for the J matrix), but we still rely on the compiler's auto-vectorizer to emit `vfmadd231pd`. An explicit intrinsics rewrite would be expected to give an additional 1.5–2× on the per-site energy difference, but the cost in maintainability and portability is real.

17 What the numbers mean

The headline number from Table 1 is $\sim 1.8 \times 10^7$ Metropolis spin-updates/sec/core for a short-range Heisenberg model on a single x86 core (up from $\sim 1.4 \times 10^7$ before this pass). For comparison, Vampire's reported figures on Skylake-class CPUs are in the range 10^7–10^8 spin-updates/sec/core depending on the model; UppASD reports $\sim 5 \times 10^7$ for similar problem sizes. We are within a factor of $\sim$ 2–3 of those codes *without* hand-rolled SIMD intrinsics. Closing the rest of the gap requires explicit AVX2/AVX-512 work on the per-site energy difference; the SoA layout (Section 4.3) is the necessary prerequisite for that, and it is now in place.

With the sublattice-coloured intra-sweep parallelism of Section 15, the same Heisenberg-honeycomb $L = 32$ kernel reaches $\sim 1.1 \times 10^8$ Metropolis updates/sec on a single 16-core socket ($T_1 \approx 0.25$ s, $S(16) \approx 6.6$, throughput $\approx 6.6 \times 1.6 \times 10^7$ u/s); the overrelaxation kernel reaches $\sim 2.0 \times 10^8$ updates/sec. This puts a single-node `ClassicalSpin_Cpp` run squarely in the upper end of the Vampire/UppASD per-node throughput range, before any explicit SIMD intrinsics work.

The cache-hierarchy effect is worth noting on its own. At $L = 32$ the spin data ($N \times d \times 8 = 49$ kB) fits in L1; at $L = 64$ (196 kB) it spills to L2; at $L = 128$ (786 kB) it spills to L3. The measured throughput drops from 1.8×10^7 to 6.5×10^6 across this range—a $\sim 3\times$ slowdown that is essentially *all memory bandwidth*. This is the regime where SIMD and the already-flat SoA layout would pay the most, and where the "flat-spin scratch" refactor we deliberately did *not* land at $L = 32$ (Section 6, last paragraph) is likely to start winning.

18 Further reading

- D. P. Landau and K. Binder, *A Guide to Monte Carlo Simulations in Statistical Physics*, 4th ed., Cambridge UP (2014).
- U. Wolff, "Collective Monte Carlo updating for spin systems", *Phys. Rev. Lett.* **62**, 361 (1989).
- R. H. Swendsen and J.-S. Wang, "Nonuniversal critical dynamics in Monte Carlo simulations", *Phys. Rev. Lett.* **58**, 86 (1987).
- K. Hukushima and K. Nemoto, "Exchange Monte Carlo method and application to spin glass simulations", *J. Phys. Soc. Jpn.* **65**, 1604 (1996).

- H. Katzgraber, S. Trebst, D. Huse, M. Troyer, "Feedback-optimized parallel tempering Monte Carlo", *J. Stat. Mech.* P03018 (2006).
- U. Drepper, "What every programmer should know about memory", `lwn.net` (2007). *Free, classic, and still essentially correct—the canonical reference for the cache-hierarchy facts above.*
- R. F. L. Evans *et al.*, "Atomistic spin model simulations of magnetic nanomaterials", *J. Phys.: Condens. Matter* **26**, 103202 (2014). *The* VAMPIRE *paper.*

Table 1: Measured single-core throughput, pinned to one CPU core (`taskset -c 0`, `OMP_NUM_THREADS=1`), gcc 13.3 with `-O3 -march=native -flto -ffp-contract=fast`. "v3" columns were captured just before the SoA bilinear table and Marsaglia sphere sampler were introduced; "v4" columns are after the trilinear pre-contraction (Section 7), the Marsaglia method extended to the mixed and strain families (Section 6), and the magnetoelastic-field fusion (Section 8) have all landed. "updates/s" is the number of *spin updates* per second; for cluster algorithms it counts "sweeps × N sites" which conflates wall time with sample quality. Numbers in bold show kernels where the v4 pass moved the needle by more than $\sim 25\,\%$.

Family	Model	Algorithm	L	N	v3 (upd/s)	v4 (upd/s)
`Lattice`	Heisenberg honeycomb	Metropolis	32	2048	1.43×10^{7}	$\mathbf{2.03\times10^{7}}$
`Lattice`	Heisenberg honeycomb	Overrelaxation	32	2048	3.35×10^{7}	3.45×10^{7}
`Lattice`	Heisenberg honeycomb	met+5over	32	2048	4.52×10^{6}	$\mathbf{5.16\times10^{6}}$
`Lattice`	Heisenberg honeycomb	Wolff	32	2048	6.83×10^{8}	5.78×10^{8}
`Lattice`	Heisenberg honeycomb	Swendsen–Wang	32	2048	2.35×10^{7}	2.43×10^{7}
`Lattice`	Kitaev honeycomb	Metropolis	32	2048	1.35×10^{7}	$\mathbf{1.86\times10^{7}}$
`Lattice`	Kitaev honeycomb	Overrelaxation	32	2048	3.21×10^{7}	3.41×10^{7}
`Lattice`	Pyrochlore	Metropolis	6	864	1.07×10^{7}	$\mathbf{1.47\times10^{7}}$
`Lattice`	Pyrochlore	Overrelaxation	6	864	2.07×10^{7}	2.45×10^{7}
`Lattice`	Pyrochlore	met+5over	6	864	2.88×10^{6}	$\mathbf{3.53\times10^{6}}$
`MixedLattice`	$TmFeO_3$ bilinear	Metropolis	4	512	1.21×10^{5}	$\mathbf{7.85\times10^{5}}$
`MixedLattice`	$TmFeO_3$ bilinear	Overrelaxation	4	512	1.50×10^{5}	$\mathbf{9.13\times10^{5}}$
`MixedLattice`	$TmFeO_3$ trilinear	Metropolis	4	512	8.20×10^{4}	$\mathbf{4.65\times10^{5}}$
`MixedLattice`	$TmFeO_3$ trilinear	Overrelaxation	4	512	9.18×10^{4}	$\mathbf{5.51\times10^{5}}$
`StrainPhonon`	Honeycomb (bilinear, $\lambda_{\mathrm{Eg}} = 0$)	Metropolis	24	1152	5.05×10^{5}	$\mathbf{4.81\times10^{6}}$
`StrainPhonon`	Honeycomb (bilinear, $\lambda_{\mathrm{Eg}} = 0$)	Overrelaxation	24	1152	5.43×10^{5}	$\mathbf{6.65\times10^{6}}$
`StrainPhonon`	Honeycomb + ME ($\lambda_{\mathrm{Eg}}\neq0$)	Metropolis	24	1152	1.04×10^{6}	$\mathbf{4.43\times10^{6}}$
`StrainPhonon`	Honeycomb + ME ($\lambda_{\mathrm{Eg}}\neq0$)	Overrelaxation	24	1152	5.64×10^{5}	$\mathbf{5.53\times10^{6}}$
Cache-hierarchy effect, Heisenberg honeycomb metropolis (v4):						
`Lattice`	Heisenberg honeycomb	Metropolis	32	2048	—	2.0×10^{7}
`Lattice`	Heisenberg honeycomb	Metropolis	64	8192	—	1.4×10^{7}
`Lattice`	Heisenberg honeycomb	Metropolis	128	32768	—	6.5×10^{6}

Table 2: Single-core LLG / spin–strain RHS-call throughput on the same machine and flags as Table 1. "rhs/s" is the number of full `ode_system()` (or `landau_lifshitz_flat()`) evaluations per second; for RK4 this is 4× the integrator step count. "v4" is just before the four ODE-targeted refactors, "v5" is after. Numbers in bold show kernels where v5 moved the needle by more than $\sim 25\,\%$.

Family	Model	L	N	v4 (rhs/s)	v5 (rhs/s)
`Lattice`	Heisenberg honeycomb (SU(2))	32	2048	1.01×10^4	$\mathbf{1.82\times10^4}$
`Lattice`	Kitaev honeycomb (SU(2))	32	2048	1.02×10^4	$\mathbf{1.83\times10^4}$
`MixedLattice`	$TmFeO_3$ bilinear (SU(2)+SU(3))	4	512	2.42×10^3	$\mathbf{3.17\times10^3}$
`MixedLattice`	$TmFeO_3$ trilinear (SU(2)+SU(3))	4	512	2.05×10^3	$\mathbf{2.58\times10^3}$
`MixedLattice`	$TmFeO_3$ driven (pulse on)	4	512	2.38×10^3	$\mathbf{3.19\times10^3}$
`StrainPhonon`	Honeycomb bilinear, λ=0	24	1152	3.45×10^3	3.42×10^3
`StrainPhonon`	Honeycomb + ME, $\lambda\neq0$	24	1152	3.31×10^3	3.34×10^3
`StrainPhonon`	Honeycomb local strain (N_{cells}=576)	24	1152	9.13×10^2	9.21×10^2